

Department of Computer and Software Engineering – ITU

CE101T: Object Oriented Programming

Course Instructor: Nadir Abbas	Dated: 16/03/2025
Teaching Assistant: Zainab, Nahal, Bilal	Semester: Spring 2025
Session: 2024-2028	Batch: BSCE2024

Assignment 8. Association and its relationships

Name	Roll number	Obtained Marks/35

Checked on: _____

Signature: _____

Submission:

- Email instructor or TA if there are any questions. You cannot look at others' solutions or use others' solutions, however, you can discuss it with each other. Plagiarism will be dealt with according to the course policy.
- Submission after due time will not be accepted.

In this assignment you have to do following tasks:

Task 1: Ensure that you have installed all three softwares in your personal computer (Github, Cygwin & CLion). Now, accept the assignment posted in classroom (e.g Google, LMS etc) and after accepting, clone the repository to your computer. Make sure you have logged into the github app with your account.

Task 2: Open Cygwin app, Move to your code directory with following command “cd <path_of_folder>”

<path_of_folder> can be automatically populated by dragging the folder and dropping it to the cygwin window.

Run the code through Cygwin, use command “make run”, to get the output of the code

Task 3: Solve the given problems, write code using **CLion** or any other IDE.

Task 4: Keep your code in the respective git cloned folder.

Task 5: Commit and Push the changes through the Github App

Task 5: Write the code in separate files (**as instructed**). Ensure that file names are in lowercase (e,g **main.cpp**).

Task 6: Run ‘**make run**’ to run C++ code

Task 7: Run ‘**make test**’ to test the C++ code

Question 1: One-to-One Association - Person and Passport

Scenario: A Person has exactly one Passport, and each Passport is assigned to only one Person.

Class Design:

Class Passport

- **Private Members:**
 - string passportNumber
 - string issueDate
 - string expiryDate
- **Public Functions:**
 - Passport(string, string, string)
 - void setPassportNumber(string) & string getPassportNumber()
 - void setIssueDate(string) & string getIssueDate()
 - void setExpiryDate(string) & string getExpiryDate()
 - bool isValid() → Checks if the passport is still valid.

Class Person

- **Private Members:**
 - string name
 - int age
 - Passport* passport
- **Public Functions:**
 - Person(string, int, Passport*)
 - void setName(string) & string getName()
 - void setAge(int) & int getAge()
 - void assignPassport(Passport*)
 - Passport* getPassport()
 - bool hasValidPassport() → Checks if the assigned passport is valid.

Question 2: One-to-Many Association - Doctor and Patients

Scenario: A Doctor can have multiple Patients, but each Patient is associated with only one Doctor.

Class Design:

Class Patient

- **Private Members:**
 - string name
 - int age
 - Doctor* assignedDoctor
- **Public Functions:**
 - Patient(string, int, Doctor*)
 - void setName(string) & string getName()

- void setAge(int) & int getAge()
- void assignDoctor(Doctor*)
- bool hasDoctor() → Checks if a doctor is assigned.

Class Doctor

- **Private Members:**
 - string name
 - string specialization
 - Patient* patients[100]
 - int patientCount
- **Public Functions:**
 - Doctor(string, string)
 - void setName(string) & string getName()
 - void setSpecialization(string) & string getSpecialization()
 - void addPatient(Patient*) & Patient** getPatients()
 - int getPatientCount() → Returns the number of assigned patients.

Question 3: Many-to-Many Association - Students and Courses

Scenario: A Student can enroll in multiple Courses, and a Course can have multiple Students.

Class Design:

Class Student

- **Private Members:**
 - string name
 - int studentID
 - Course* courses[10]
 - int courseCount
- **Public Functions:**
 - Student(string, int)
 - void enrollCourse(Course*) & Course** getCourses()
 - bool isEnrolledInCourse(Course*) → Checks if a student is enrolled in a specific course.

Class Course

- **Private Members:**
 - string title
 - int courseID
 - Student* students[50]
 - int studentCount
- **Public Functions:**
 - Course(string, int)

- void enrollStudent(Student*) & Student** getStudents()
- int getEnrollmentCount() → Returns the number of enrolled students.

Question 4: Multiple Associations - Teacher, Classroom, and Students

Scenario: A Teacher is assigned to only one Classroom (One-to-One), but a Classroom can have multiple Students (One-to-Many).

Class Design:

Class Students

- **Private Members:**
 - string name
 - int studentID
- **Public Functions:**
 - Students(string, int)
 - bool isAssignedToClassroom(Classroom*)

Class Classroom

- **Private Members:**
 - int classID
 - Students* students[30]
 - int studentCount
- **Public Functions:**
 - Classroom(int)
 - void addStudent(Students*) & Students** getStudents()
 - int getTotalStudents()

Class Teacher

- **Private Members:**
 - string name
 - Classroom* assignedClassroom
- **Public Functions:**
 - Teacher(string, Classroom*)
 - void assignClassroom(Classroom*) & Classroom* getClassroom()
 - bool hasClassroom()

Question 5: Association with Different Roles - Company, Employees, and Projects

Scenario: A Company has multiple Employees (One-to-Many). Each Employee can be assigned to a single Project, but a Project can have multiple Employees (Many-to-Many).

Class Design:

Class Employee

- **Private Members:**
 - string name
 - int employeeID
 - Project* project
- **Public Functions:**
 - Employee(string, int, Project*)
 - void assignProject(Project*) & Project* getProject()
 - bool isAssignedToProject()

Class Project

- **Private Members:**
 - string title
 - int projectID
 - Employee* employees[20]
 - int employeeCount
- **Public Functions:**
 - Project(string, int)
 - void addEmployee(Employee*) & Employee** getEmployees()
 - int getTotalEmployees()

Class Company

- **Private Members:**
 - string name
 - Employee* employees[100]
 - int employeeCount
- **Public Functions:**
 - Company(string)
 - void addEmployee(Employee*) & Employee** getEmployees()
 - int getTotalEmployees()

Question 6: UML Diagram - Restaurant, Menu, and Dish (Composition) & Hospital, Doctors, and Patients (Aggregation)

Task:

- Draw the UML diagram for the **Restaurant, Menu, and Dish** (Composition) example.
- Draw the UML diagram for the **Hospital, Doctors, and Patients** (Aggregation) example.

UML Diagram Requirements:

- Show class names, attributes, and functions.

- Use correct relationships.
- Make UML diagrams in a single image.
- Submit as an image file (e.g., UML_Diagram.png).

Assessment Rubric for Assignment

Performance metric	CL O	Able to complete the task over 80% (4-5)	Able to complete the task 50-80% (2-3)	Able to complete the task below 50% (0-1)	Marks
1. Realization of experiment	3	Executes without errors excellent user prompts, good use of symbols, spacing in output. Through testing has been completed.	Executes without errors, user prompts are understandable, minimum use of symbols or spacing in output. Some testing has been completed.	Does not execute due to syntax errors, runtime errors, user prompts are misleading or non-existent. No testing has been completed.	
2. Conducting experiment	2	Able to make changes and answered all questions.	Partially able to make changes and few incorrect answers.	Unable to make changes and answer all questions.	
3. Computer use	4	Document submission timely.	Document submission late.	Document submission not done.	
4. Teamwork	4	Actively engages and cooperates with other group member(s) in effective manner.	Cooperates with other group member(s) in a reasonable manner but conduct can be improved.	Distracts or discourages other group members from conducting the experiment	
5. Laboratory safety and disciplinary rules	2	Code comments are added and does help the reader to understand the code.	Code comments are added and does not help the reader to understand the code.	Code comments are not added.	
6. Data collection	2	Excellent use of white space, creatively organized work, excellent use of variables and constants, correct identifiers for constants, No line-wrap.	Includes name, and assignment, white space makes the program fairly easy to read. Title, organized work, good use of variables.	Poor use of white space (indentation, blank lines) making code hard to read, disorganized and messy.	
7. Data analysis	3	Solution is efficient, easy to understand, and maintain.	A logical solution that is easy to follow but it is not the most efficient.	A difficult and inefficient solution.	
Total (out of 35):					